

Java Strings

Fundamentals and Usage

A Comprehensive Guide to

“ String Manipulation in Java ”

Java Programming Lecture Series

What are **Strings** in Java?

Definition & Characteristics

- ✓ A **String** is a sequence of characters
- ✓ Part of **java.lang** package (automatically imported)
- ✓ One of the most commonly used classes in Java
- ✓ **Immutable**: Once created, cannot be changed
- ✓ Implements **CharSequence**, **Comparable**, and **Serializable** interfaces

String Declaration

```
String message = "Hello, Java!";  
// String is a reference type, not primitive
```

Internal Representation



Character array (char[])

Key Points to Remember

- 💡 Strings are objects, not primitives, and are handled differently from other data types in Java. Understanding their immutable nature is crucial for efficient string manipulation.

Creating **Strings** in Java

Using String Literals

```
String name = "John Doe";  
// Stored in String Pool
```

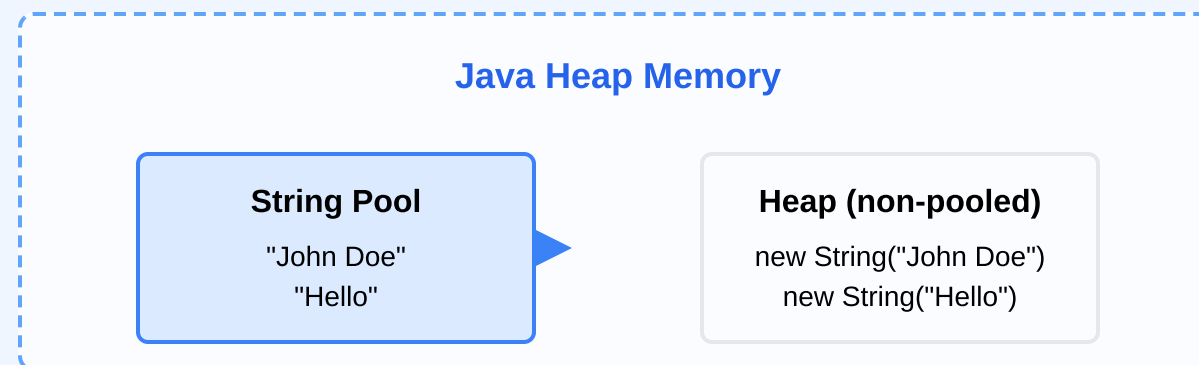
- ❖ Most common and efficient method
- ❖ Uses the **String Pool** for memory optimization
- ❖ Identical literals reference the same object

Using the new Keyword

```
String name = new String("John Doe");  
// Creates object in heap memory
```

- ❖ Forces creation of a new String object
- ❖ Stored in heap memory (outside String Pool)
- ❖ Less memory-efficient than literals

The String Pool



- 💡 The String Pool is a special memory area in the heap. When you create a string literal, Java first checks if an identical string exists in the pool. If it does, the reference is reused; if not, a new string is added to the pool.

The **Immutable** Nature of Strings

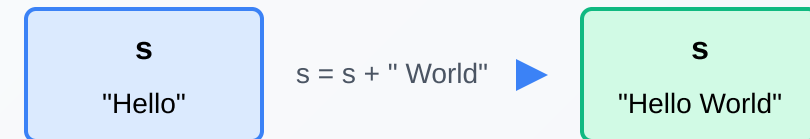
What is Immutability?



Once a String object is created, its content **cannot be changed**.

```
String s = "Hello";  
s = s + " World"; // Creates new String  
// s now references "Hello World"  
// Original "Hello" still exists
```

String Modification Visualization



A new String object is created and the reference is updated

Benefits of Immutability

-  **Security:** Cannot be altered once created
-  **Thread Safety:** Safe for concurrent operations
-  **HashCode Caching:** Improves performance in collections
-  **String Pool:** Enables memory optimization

Performance Implications



Frequent string modifications can lead to performance issues

```
String result = "";  
for (int i = 0; i < 10000; i++) {  
    result += i; // Creates 10,000 String  
objects!  
}
```

Use **StringBuilder** or **StringBuffer** for frequent modifications

Common Operations: Length & Concatenation

Getting String Length

length()

Returns the number of characters in the string

```
String text = "Hello, Java!";  
int length = text.length(); // 12
```

💡 Empty strings have a length of 0, not null

String Concatenation

+ Operator

Most common way to join strings

```
String firstName = "John";  
String lastName = "Doe";  
String fullName = firstName + " " +  
lastName;  
// "John Doe"
```

concat()

Method alternative to + operator

```
String greeting = "Hello, ";  
String name = "Java!";  
String message = greeting.concat(name);  
// "Hello, Java!"
```

Concatenation with Non-String Types

```
int age = 30;  
String message = "I am " + age + " years old.";  
// "I am 30 years old."
```

When using the + operator with a String and any other type:



- The non-String value is automatically converted to a String
- The conversion uses the toString() method or String.valueOf()
- This works with primitives (int, double, etc.) and objects

Common Operations: Accessing Characters

Accessing Individual Characters

`charAt(int index)`

Returns the character at the specified index

```
String text = "Java";  
char firstChar = text.charAt(0); // 'J'  
char lastChar = text.charAt(3); // 'a'
```



Throws `StringIndexOutOfBoundsException` if index is negative or $\geq$ length

String Indexing

J	a	v	a
0	1	2	3

Indices start at 0

Converting to character array:

```
String text = "Java";  
char[] charArray = text.toCharArray();  
// charArray = {'J', 'a', 'v', 'a'}
```

Extracting Substrings

`substring(int beginIndex)`

Returns substring from beginIndex to end

```
String text = "Hello, World!";  
String sub1 = text.substring(7);  
// "World!"
```

`substring(int beginIndex, int endIndex)`

Returns substring from beginIndex to endIndex-1

```
String text = "Hello, World!";  
String sub2 = text.substring(0, 5);  
// "Hello"
```

i The substring methods create new String objects (remember immutability). The original String remains unchanged.

String Comparison: equals() vs ==

Content Comparison

equals(Object obj)
Compares the content (characters) of strings

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");

s1.equals(s2); // true
s1.equals(s3); // true
```

equalsIgnoreCase(String str)
Case-insensitive content comparison

```
"Hello".equalsIgnoreCase("hello"); // true
"Hello".equals("hello"); // false
```

Reference Comparison

== Operator
Compares object references (memory addresses)

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");

s1 == s2; // true (same reference in String Pool)
s1 == s3; // false (different references)
```

⚠️ Never use == to compare String content in Java!

Comparison Summary

Feature	equals()	== Operator
Compares	Content (character sequence)	Memory references
Use for String comparison	✔️ Recommended	❌ Not recommended
Handles String Pool optimization	✔️ Yes	❌ Unpredictable results
Null safety	❌ Throws NullPointerException if null	✔️ Can compare with null

String Comparison: `compareTo()`

Lexicographical Comparison

`compareTo(String anotherString)`

Compares strings lexicographically (dictionary order)

```
"apple".compareTo("banana"); // negative
"banana".compareTo("apple"); // positive
"apple".compareTo("apple"); // 0
```



Returns an integer value indicating the relationship:

Negative: this string comes before the argument
Zero: strings are equal
Positive: this string comes after the argument

Case-Insensitive Comparison

`compareToIgnoreCase(String str)`

Case-insensitive lexicographical comparison

```
"Apple".compareToIgnoreCase("apple"); // 0
"Apple".compareTo("apple"); // negative
// (uppercase comes before lowercase in ASCII)
```



Useful for sorting strings in a case-insensitive manner

Common Use Cases

Sorting Strings

```
List<String> names = new ArrayList<>();
names.add("Charlie");
names.add("Alice");
names.add("Bob");

Collections.sort(names); // Uses compareTo()
// Result: ["Alice", "Bob", "Charlie"]
```

Custom Sorting

```
List<String> names = new ArrayList<>();
// ... add names

Collections.sort(names, (String s1, String s2)
-> {
    return s1.compareToIgnoreCase(s2);
});
// Case-insensitive sorting
```



The `String` class implements the `Comparable` interface, which requires the `compareTo()` method. This enables strings to be used in sorted collections and with sorting algorithms.

Searching Within Strings

Finding Substrings

`indexOf(String str)`

Returns the index of the first occurrence

```
String text = "Hello, World!";
int index = text.indexOf("World"); // 7
int notFound = text.indexOf("Java"); // -1
```

`lastIndexOf(String str)`

Returns the index of the last occurrence

```
String text = "Hello, Hello!";
int index = text.lastIndexOf("Hello"); // 7
```

Checking for Substrings

`contains(CharSequence s)`

Checks if string contains the specified sequence

```
String text = "Hello, World!";
boolean contains = text.contains("World");
// true
```

`startsWith(String prefix)`

Checks if string starts with the specified prefix

```
String text = "Hello, World!";
boolean starts = text.startsWith("Hello");
// true
```

`endsWith(String suffix)`

Checks if string ends with the specified suffix

```
String text = "Hello, World!";
boolean ends = text.endsWith("!"); // true
```

Advanced Search Options

`indexOf(String str, int fromIndex)`

Searches from the specified index

```
String text = "Java Java Java";
int index = text.indexOf("Java", 5); // 5
```

`matches(String regex)`

Checks if string matches the regex pattern

```
String email = "user@example.com";
boolean isEmail = email.matches(".*@.*\\..*");
// Basic email validation
```



All search methods return meaningful values: index positions for `indexOf()`/`lastIndexOf()` or boolean results for `contains()`/`startsWith()`/`endsWith()`. When a substring is not found, index methods return **-1**.

Modifying Strings: Case & Trimming

Case Conversion

toLowerCase()

Converts all characters to lowercase

```
String text = "Hello, WORLD!";
String lower = text.toLowerCase();
// "hello, world!"
```

toUpperCase()

Converts all characters to uppercase

```
String text = "Hello, World!";
String upper = text.toUpperCase();
// "HELLO, WORLD!"
```



Case conversion is useful for case-insensitive comparisons and standardizing user input

Trimming Whitespace

trim()

Removes leading and trailing whitespace

```
String text = " Hello, World! ";
String trimmed = text.trim();
// "Hello, World!"
```

strip() (Java 11+)

Similar to trim() but Unicode-aware

```
String text = " Hello, World! ";
String stripped = text.strip();
// "Hello, World!"
```



Java 11+ also provides [stripLeading\(\)](#) and [stripTrailing\(\)](#) for more control

Remember: Immutability!

All these methods return **new String objects**. The original String remains unchanged.



```
String original = " Hello ";
original.trim(); // This has no effect on original!
System.out.println(original); // Still prints " Hello "
```

```
String trimmed = original.trim(); // Correct usage
System.out.println(trimmed); // Prints "Hello"
```

Modifying Strings: Replacing Content

Character Replacement

replace(char oldChar, char newChar)

Replaces all occurrences of a character

```
String text = "Hello, World!";
String replaced = text.replace('l', '*');
// "He**o, Wor*d!"
```

 Returns the original string if no matches are found

String Replacement

replace(CharSequence target, CharSequence replacement)

Replaces all occurrences of a substring

```
String text = "Java is cool. Java is powerful.";
String replaced = text.replace("Java", "Python");
// "Python is cool. Python is powerful."
```

 Literal replacement, not regex-based

Regular Expression Replacement

replaceAll(String regex, String replacement)

Replaces all matches of regex pattern

```
String text = "Java8 is Java11's predecessor";
String replaced = text.replaceAll("\\d+", "X");
// "JavaX is JavaX's predecessor"
```

replaceFirst(String regex, String replacement)

Replaces only the first match

```
String text = "Java8 is Java11's predecessor";
String replaced = text.replaceFirst("Java", "Python");
// "Python8 is Java11's predecessor"
```

Regular expressions are powerful but can be complex:



- Special characters need to be escaped with `\\` in Java strings
- Performance can be impacted with complex patterns
- Use `replace()` for simple literal replacements
- Use `replaceAll()`/`replaceFirst()` when pattern matching is needed

Splitting and Joining Strings

Splitting Strings

`split(String regex)`

Splits string around matches of the regex

```
String text = "apple,banana,orange";
String[] fruits = text.split(",");
// fruits = {"apple", "banana", "orange"}
```

 You can limit the number of splits with `split(String regex, int limit)`

Joining Strings

`String.join(CharSequence delimiter, CharSequence... elements)`

Joins elements with the delimiter (Java 8+)

```
String[] fruits = {"apple", "banana", "orange"};
String joined = String.join(", ", fruits);
// "apple, banana, orange"
```

 Also works with collections: `String.join(", ", myList)`

Common Use Cases

Parsing CSV Data

```
String csvLine = "John,Doe,30,New York";
String[] data = csvLine.split(",");
// Process each field
String firstName = data[0]; // "John"
String lastName = data[1]; // "Doe"
```

Building Comma-Separated Lists

```
List<String> tags = new ArrayList<>();
tags.add("java");
tags.add("programming");
tags.add("strings");

String tagList = String.join(", ", tags);
// "java, programming, strings"
```

Be careful with `split()` when using regex special characters:



```
// This won't work as expected:
"1.2.3.4".split("."); // "." is a regex special character

// Correct way:
"1.2.3.4".split("\\."); // Escape the dot
```

String Formatting

String.format() Method

String.format(String format, Object... args)

Returns a formatted string using the specified format string and arguments

```
String name = "John";
int age = 30;
String formatted = String.format(
    "Name: %s, Age: %d", name, age);
// "Name: John, Age: 30"
```

printf-Style Formatting

System.out.printf(String format, Object... args)

Prints a formatted string to the console

```
double price = 19.99;
System.out.printf(
    "Price: %.2f", price);
// Prints "Price: $19.99"
```

💡 Same format specifiers as String.format()

Common Format Specifiers

%s

String

```
String.format("%s", "Hello")
// "Hello"
```

%d

Integer

```
String.format("%d", 42)
// "42"
```

%f

Float/Double

```
String.format("%.2f", 3.14159)
// "3.14"
```

%b

Boolean

```
String.format("%b", true)
// "true"
```

%c

Character

```
String.format("%c", 'A')
// "A"
```

%n

Line separator

```
String.format("Line1%nLine2")
// "Line1\nLine2"
```

Format modifiers provide additional control:



- **%10s** - Right-justified in 10-character field
- **%-10s** - Left-justified in 10-character field
- **%.2f** - Float with 2 decimal places
- **%,d** - Integer with thousands separator (e.g., 1,000,000)

Mutable Alternatives: StringBuilder & StringBuffer

The Problem with String Concatenation

```
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i; // Creates 10,000 String objects!
}
```

 Inefficient for frequent modifications due to String immutability

The Solution: Mutable String Classes

StringBuilder

Mutable, not thread-safe, faster

StringBuffer

Mutable, thread-safe, slightly slower

 Both provide similar methods for string manipulation

Using StringBuilder

```
StringBuilder sb = new StringBuilder();

// Append various types
sb.append("Hello");
sb.append(" ");
sb.append("World");
sb.append("! Count: ");
sb.append(42);

// Convert back to String
String result = sb.toString();
// "Hello World! Count: 42"
```

Common Methods

Method	Description
<code>append()</code>	Adds text at the end
<code>insert(int offset, String str)</code>	Inserts text at specified position
<code>delete(int start, int end)</code>	Removes characters in range
<code>replace(int start, int end, String str)</code>	Replaces characters in range
<code>reverse()</code>	Reverses character sequence
<code>toString()</code>	Converts to String

Performance Comparison

For frequent string modifications:

- 
- StringBuilder:** Use for single-threaded scenarios (most common)
 - StringBuffer:** Use only when thread safety is required
 - String concatenation:** Use only for simple, one-time operations

Note: Modern Java compilers may automatically optimize some string concatenation operations to use StringBuilder behind the scenes, but explicit use is still recommended for loops and complex operations.

Best Practices & Summary

String Best Practices

Respect Immutability

Always capture the result of string operations

```
// Wrong
str.toUpperCase(); // No effect!

// Correct
str = str.toUpperCase();
```

Use StringBuilder for Concatenation

Especially in loops or when building large strings

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i);
}
String result = sb.toString();
```

More Best Practices

Use equals() for Comparison

Never use == for string content comparison

```
// Wrong
if (str1 == str2) { ... }

// Correct
if (str1.equals(str2)) { ... }
```

Handle Null Values

Check for null before calling string methods

```
// Defensive programming
if (str != null && str.length() > 0) {
    // Safe to use str
}
```

Key Takeaways

Immutability

Strings cannot be changed after creation; operations return new String objects

Rich API

Java provides numerous methods for string manipulation, comparison, and searching

Performance Awareness

Choose appropriate tools (String vs StringBuilder) based on your use case



Understanding String handling is fundamental to Java programming. Proper string manipulation improves code readability, maintainability, and performance.

Questions & Answers



Any questions about Java Strings?

Thank You!

For additional resources on Java String handling:



Java Documentation



Practice Exercises



Tutorial Videos